```
            _events.Append(rejected);
            _events.Append(commandIssuedEvent);
        }
    }
```

In this example, the outbox relay will have to execute the commands on relevant endpoints for each instance of `CommandIssuedEvent`. As in the case of publishing domain events, separating the transition of the saga's state from the execution of commands ensures that the commands will be executed reliably, even if the process fails at any stage.

Consistency

Although the saga pattern orchestrates a multicomponent transaction, the states of the involved components are eventually consistent. And although the saga will eventually execute the relevant commands, no two transactions can be considered atomic. This correlates with another aggregate design principle:

> *Only the data within an aggregate's boundaries can be considered strongly consistent. Everything outside is eventually consistent.*

Use this as a guiding principle to make sure you are not abusing sagas to compensate for improper aggregate boundaries. Business operations that have to belong to the same aggregate require strongly consistent data.

The saga pattern is often confused with another pattern: process manager. Although the implementation is similar, these are different patterns. In the next section, we'll discuss the purpose of the process manager pattern and how it differs from the saga pattern.

Process Manager

The saga pattern manages simple, linear flow. Strictly speaking, a saga matches events to the corresponding commands. In the examples we used to demonstrate saga implementations, we actually implemented simple matching of events to commands:

- `CampaignActivated` event to `PublishingService.SubmitAdvertisement` command
- `PublishingConfirmed` event to `Campaign.TrackConfirmation` command
- `PublishingRejected` event to `Campaign.TrackRejection` command

The process manager pattern, shown in Figure 9-14, is intended to implement a business-logic-based process. It is defined as a central processing unit that maintains the state of the sequence and determines the next processing steps.[2]

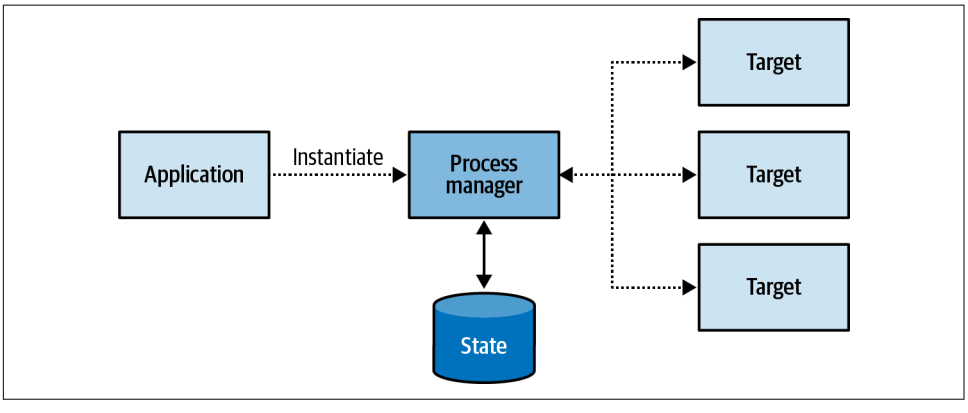

Figure 9-14. Process manager

As a simple rule of thumb, if a saga contains if-else statements to choose the correct course of action, it is probably a process manager.

Another difference between a process manager and a saga is that a saga is instantiated implicitly when a particular event is observed, as in `CampaignActivated` in the preceding examples. A process manager, on the other hand, cannot be bound to a single source event. Instead, it's a coherent business process consisting of multiple steps. Hence, a process manager has to be instantiated explicitly. Consider the following example:

Booking a business trip starts with the routing algorithm choosing the most cost-effective flight route and asking the employee to approve it. In case the employee prefers a different route, their direct manager needs to approve it. After the flight is booked, one of the preapproved hotels has to be booked for the appropriate dates. If no hotels are available, the flight tickets have to be canceled.

In this example, there is no central entity to trigger the trip booking process. The trip booking is the process and it has to be implemented as a process manager (see Figure 9-15).

2 Hohpe, G., & Woolf, B. (2003). *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley.

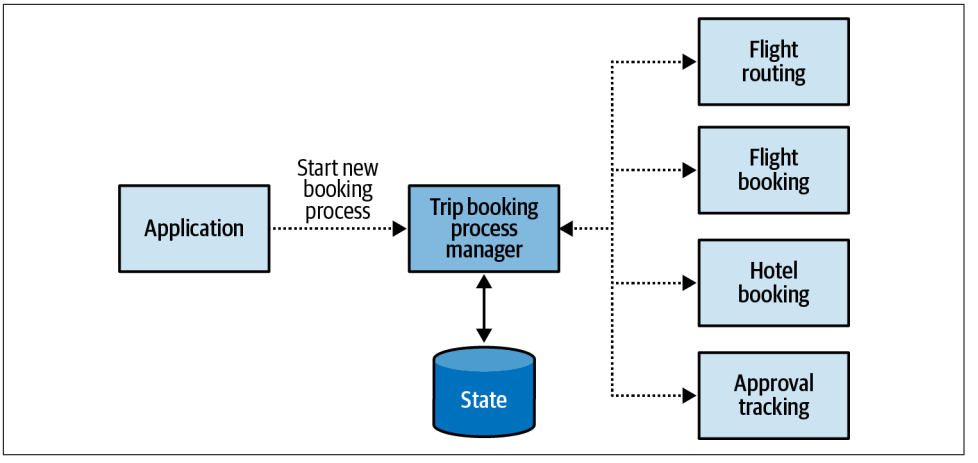

Figure 9-15. Trip booking process manager

From an implementation perspective, process managers are often implemented as aggregates, either state based or event sourced. For example:

```
public class BookingProcessManager
{
    private readonly IList<IDomainEvent> _events;
    private BookingId _id;
    private Destination _destination;
    private TripDefinition _parameters;
    private EmployeeId _traveler;
    private Route _route;
    private IList<Route> _rejectedRoutes;
    private IRoutingService _routing;
    ...

    public void Initialize(Destination destination,
                           TripDefinition parameters,
                           EmployeeId traveler)
    {
        _destination = destination;
        _parameters = parameters;
        _traveler = traveler;
        _route = _routing.Calculate(destination, parameters);

        var routeGenerated = new RouteGeneratedEvent(
            BookingId: _id,
            Route: _route);

        var commandIssuedEvent = new CommandIssuedEvent(
            command: new RequestEmployeeApproval(_traveler, _route)
        );

        _events.Append(routeGenerated);
```

```
        _events.Append(commandIssuedEvent);
    }

    public void Process(RouteConfirmed confirmed)
    {
        var commandIssuedEvent = new CommandIssuedEvent(
            command: new BookFlights(_route, _parameters)
        );

        _events.Append(confirmed);
        _events.Append(commandIssuedEvent);
    }

    public void Process(RouteRejected rejected)
    {
        var commandIssuedEvent = new CommandIssuedEvent(
            command: new RequestRerouting(_traveler, _route)
        );

        _events.Append(rejected);
        _events.Append(commandIssuedEvent);
    }

    public void Process(ReroutingConfirmed confirmed)
    {
        _rejectedRoutes.Append(route);
        _route = _routing.CalculateAltRoute(destination,
                                            parameters, rejectedRoutes);
        var routeGenerated = new RouteGeneratedEvent(
            BookingId: _id,
            Route: _route);

        var commandIssuedEvent = new CommandIssuedEvent(
            command: new RequestEmployeeApproval(_traveler, _route)
        );

        _events.Append(confirmed);
        _events.Append(routeGenerated);
        _events.Append(commandIssuedEvent);
    }

    public void Process(FlightBooked booked)
    {
        var commandIssuedEvent = new CommandIssuedEvent(
            command: new BookHotel(_destination, _parameters)
        );

        _events.Append(booked);
        _events.Append(commandIssuedEvent);
    }
```

```
    ...
}
```

In this example, the process manager has its explicit ID and persistent state, describing the trip that has to be booked. As in the earlier example of a saga pattern, the process manager subscribes to events that control the workflow (`RouteConfirmed`, `RouteRejected`, `ReroutingConfirmed`, etc.), and it instantiates events of type `CommandIssuedEvent` that will be processed by an outbox relay to execute the actual commands.

Conclusion

In this chapter, you learned the different patterns for integrating a system's components. The chapter began by exploring patterns for model translations that can be used to implement anticorruption layers or open-host services. We saw that translations can be handled on the fly or can follow a more complex logic, requiring state tracking.

The outbox pattern is a reliable way to publish aggregates' domain events. It ensures that domain events are always going to be published, even in the face of different process failures.

The saga pattern can be used to implement simple cross-component business processes. More complex business processes can be implemented using the process manager pattern. Both patterns rely on asynchronous reactions to domain events and the issuing of commands.

Exercises

1. Which bounded context integration pattern requires implementation of model transformation logic?
   a. Conformist
   b. Anticorruption layer
   c. Open-host service
   d. B and C
2. What is the goal of the outbox pattern?
   a. Decouple messaging infrastructure from the system's business logic layer
   b. Reliably publish messages
   c. Support implementation of the event-sourced domain model pattern
   d. A and C